ParcoursSup

A Problem-Based Learning Activity (PBL) for students of the Advanced Algorithmics 1 module

Henri HAGBE

Problem Scenario

Tomorrow morning, an inquiry commission will release a public report entitled: “ParcourSup lacks transparency and reproduces inequalities on a large scale.” The opening lines of this report will cause an uproar.

Media pressure escalates when a member of parliament calls for a parliamentary inquiry commission. As an emergency measure, the Ministry of Higher Education mandates an “Algo + Audit” team.

This is where you come in.

You are a team of developers working for the public sector. Your mission: produce a credible prototype capable of recalculating, at large scale, the call orders for programs (i.e., the order in which admission offers would be sent), then demonstrate through tests that your version is more robust and more auditable than what currently exists.

Note: As you have experienced yourselves, there is no “single national ranking” of students. Each program has its own list of applicants. However, the calculation of call orders is a national massive: thousands of lists to produce over huge volumes, in a limited time.

Required Work

Your mission is therefore to design a tool that can:

1. **Load data:**
    - Read data from a CSV file containing all possible information on the wishes of several hundred thousand students
    - Insert the data into a suitable structure to facilitate manipulation and searching

2. **Format the data correctly:**
    - Implement intermediate functions to:
        - Organise data in a clear and understandable structure.
        - Display the complete data for a given application.
        - Retrieve the information of a given student using their identifier.

3. **Sort and display data:**
    - Implement and adapt the sorting algorithms from the course to rank students' wishes by chosen program.

4. **Understand and replicate the ParcoursSup mechanism (simplified!)**
    - Group applications by program (program_id), calculate a call order per program, then extract accepted applicants (capacity) + waiting list.

5. **Propose 2 POLICIES**
    - A simple and auditable sorting baseline on the provided data.
    - A corrected version aimed at reducing a measurable flaw (arbitrary ties, incorrectly calculated micro-gaps, non-optimal performance).

6. **Implement a sorting strategy suited to large-scale data** (merge sort / quicksort / bucket sort if relevant) and measure performance on large volumes.

7. **Perform audit tests on at least 2 system flaws** (before/after correction) and produce quantified evidence:

    - Que se passe-t-il en cas de ...
        - Arbitrary ties (implicit tie-break)
        - Overly fine micro-gaps (to the hundredth of a point)
        - Unrealistic performance (an $O(n^2)$ sort for millions of records!)

8. **Prepare a short report (2–4 pages) and an oral presentation (format: "hearing before a commission").**

9. **[Bonus] Compare sorting algorithms with each other:**
    - Allow testers to:
        - Conduct a comparative study of sorting algorithms (merge, quick, bucket and insertion) with graphical output using the Python library ***matplotlib***. Students will plot a curve of the duration of the different sorts as a function of the size of the data to be sorted.

Provided Data: Structure and Meaning of Fields

Each row represents an application (a wish): a candidate applies to a program. The objective is to produce a call order per program.

candidate_id : Candidate identifier (integer). Also used as the final tie-break (repeatable, unambiguous).

program_id : Identifiant de la formation (entier). Permet de regrouper les candidatures par formation.

score : Application score (bounded integer, e.g. 0..10000). Used for ranking (baseline); bounded, which is useful for bucket sort.

timestamp : Order/time of submission of the wish (integer). Natural tie-break for auditing: at equal score, who was “ahead”?

is_scholarship : 0/1: scholarship student (means-tested grant). Used to test fairness rules (quota, priority on tie) and to measure impact (% scholarship students admitted vs % scholarship students applied).

hs_id : Identifier of the student’s secondary school (code). Used for statistical auditing: diversity of schools represented among admitted students, concentration on a few schools, etc. (an analysis tool, not a value judgment).

Resources for Addressing the Problem Scenario

- Course 3 and Lab 3 « Sorting algorithms: Insertion Sort, Selection Sort, Bubble Sort ».
- Course 4 and Lab 4 « Sorting algorithms 2 : Merge Sort, Quicksort)
- Course on « file » (1st semester : « Python Language »)
- The dataset available via the link and on Moodle

Learning Objectives of the PBL (ILO):

- Understand and master sorting algorithms.
- Test and compare sorting algorithms and be able to identify the fastest one.
- Distribute roles and tasks within the PBL group.
- Comment on programs in a relevant and meaningful way.

- Synthesise the group’s work.
- Use Python to handle a large-scale project on government data.
- Import, use and test a dataset from a CSV file.
- Understand the link between the theoretical complexity calculated in class and the actual time taken by the different sorting algorithms studied this year